

Что такое DevOps?

Правдин Иван

24.05.2025

Промышленная разработка ПО:

1. **Организационно-процессная** составляющая
2. **Техническая** составляющая

Между разработкой и эксплуатацией находится важный, но часто ``**невидимый**'' слой — именно он в центре нашего курса

Зачем новые подходы?

Индустрия разработки ПО изменилась:

- Программы стали **сложнее**

Индустрия разработки ПО изменилась:

- Программы стали **сложнее**
- Работают как **веб-сервисы 24/7**

Индустрия разработки ПО изменилась:

- Программы стали **сложнее**
- Работают как **веб-сервисы 24/7**
- Бизнес требует **быстрых обновлений**

Индустрия разработки ПО изменилась:

- Программы стали **сложнее**
- Работают как **веб-сервисы 24/7**
- Бизнес требует **быстрых обновлений**
- Раньше: релиз раз в несколько месяцев

Индустрия разработки ПО изменилась:

- Программы стали **сложнее**
- Работают как **веб-сервисы 24/7**
- Бизнес требует **быстрых обновлений**
- Раньше: релиз раз в несколько месяцев
- Сейчас: обновления каждую неделю или чаще

Что требует такая скорость?

- **Высокая автоматизация**
- **Тесное взаимодействие** между командами
- **Быстрое внесение** и проверка изменений

Традиционные методы с длительными циклами и раздельной работой команд **не успевают**

Почему мы здесь?

Сегодня любой инженер — разработчик, тестировщик, админ — не может качественно работать, не понимая:

- Как код **тестируется**
- Как код **деплоится**
- Как код **масштабируется**
- Как код **мониторится**

Цель курса: понять современный процесс разработки и практики, делающие его быстрым, надёжным и гибким

1. Введение в DevOps: философия и культура
2. Системы контроля версий и Git workflows
3. CI/CD: концепции и инструменты
4. Инфраструктура как код (IaC)
5. Контейнеризация и оркестрация
6. Мониторинг и логирование
7. DevSecOps и безопасность в пайплайнах
8. Deployment стратегии и практический проект

Но лучше всего понять эти проблемы
на **конкретных примерах**

Готовы к историям из жизни IT-команд?

Кейс 1: Пятничный релиз

Пятница. Вечер. В конце рабочего дня разработчик вручную выкатывает билд на прод. Через час пользователи жалуются на баги, а восстановить прошлую версию невозможно: она только на ноуте разработчика, который уже уехал.

Что делать?

1. Нужно заставить Васю лучше тестировать
2. Надо было подождать до понедельника
3. Не хватает процессов и автоматизации
4. Баги — это нормально, в понедельник починим

Кейс 2: Медленная работа после релиза

Сисадмин Петя устанавливает новую версию приложения на прод. Предварительно разработчик Вася и тестировщик Саша тестировали эту версию на стейджинге. После релиза пользователи жалуются: приложение начало работать заметно медленнее. Оказывается, дело в новом функционале.

Что делать?

1. Нужно лучше тестировать, а лучше нанять ещё 2 QA инженеров
2. Автоматизировать настройку серверов
3. Купить более мощные сервера для продакшена
4. Вася должен писать больше тестов

Видите закономерность?

В обоих случаях проблема в **ручных процессах**
и отсутствии **автоматизации**

Но DevOps решает не только технические проблемы...

Кейс 3: Закончилось место на диске

Разработчик Вася сделал новую фичу, которая увеличивает использование диска. Сисадмин Петя не знал про новую фичу. Через два дня приложение падает — место закончилось.

Что делать?

1. Ввести правило: все новые фичи должны согласовываться с Ops за 3 дня
2. Закупить ``железо`` про запас
3. Настроить мониторинг и квотирование
4. Петя должен смотреть все ченжлоги перед релизом

Кейс 5: Важная фича и окно релиза

Разработчик Вася за два дня сделал фичу, которая, предположительно, принесет много новых клиентов компании. При этом ближайшее окно релиза через 3 недели.

Что делать?

1. Разрешить релизить важные фичи вне окна релиза
2. Мобилизовать тестировщиков с других активностей
3. Ввести ежедневные планерки для пересмотра приоритетов
4. Вообще убрать окна релиза и релизить по желанию

Автоматизация — это хорошо,
коммуникация — тоже

Но есть еще одна важная проблема

Что делать, когда что-то идет не так?

Неожиданно сайт перестает работать. Выяснилось, что кто-то удалил таблицу users в базе данных. Резервная копия устарела, логи не велись.

Что делать?

1. Разрешить работать с prod базой только ``роботам``
2. Поменять все пароли и сообщить их только некоторым синьорам
3. Вести список ``провинившихся`` разработчиков
4. Нанять отдельного специалиста DBA

Кейс 7: Вася ушел в отпуск

Вася завершил выпуск новой функции приложения, но на следующий день ушёл в планируемый отпуск. Остальные сотрудники не знают, как её тестировать, настраивать и использовать, из-за отсутствия документации.

Что делать?

1. Внести в общие правила пункт ``Не уходить в отпуск без документации``
2. Нанять отдельного техписателя для написания документации
3. Ввести в KPI пункт ``покрытие документацией``
4. Задача выполнена только тогда, когда она в prod и протестирована

Во всех случаях проблема не в людях

Не в том, что Вася плохо программирует
или Петя невнимательный

Проблема в организации работы

1. Ручные процессы (кейсы 1, 2, 5)

- Все делается руками вместо автоматизации
- Результат: ошибки и нестабильность

2. Изоляция команд (кейс 3)

- Dev и Ops не понимают друг друга
- Результат: проблемы коммуникации

3. Неправильная культура (кейсы 6, 7)

- Ищут виноватых вместо решения проблем
- Результат: повторение ошибок

DevOps решает все три проблемы:

- **Автоматизация** вместо ручной работы
- **Сотрудничество** вместо изоляции команд
- **Культура обучения** вместо поиска виноватых

Вопрос: если DevOps - это культура для всех,
то зачем существует должность
``DevOps инженер``?

DevOps — это культура и практики для всей команды

Но на практике кто-то должен:

- Настраивать CI/CD пайплайны
- Управлять инфраструктурой
- Внедрять практики автоматизации
- Быть связующим звеном между Dev и Ops

DevOps инженер — специалист, который помогает командам правильно применять DevOps практики

Как при строительстве дома:

Каждый рабочий знает свое дело —
электрик, сантехник, каменщик

Но есть **прораб**, который координирует всех,
следит за процессом, решает проблемы
между специалистами

DevOps инженер — прораб в IT-проекте

Запомните:

DevOps инженер не делает DevOps за всех

Он **координирует процесс** и помогает
команде работать слаженно

Прораб не заменяет электрика,
но следит, чтобы работа всех специалистов
не мешала друг другу

- **Конкретные инструменты:** как автоматизировать процессы
- **Практики сотрудничества:** как организовать взаимодействие команд
- **Культурные изменения:** как внедрить правильную культуру

Вопросы?